

CS-4112: Introduction to Machine Learning

Activation Functions

Arbish Akram

Department of Computer Science
Government College University

Slides by Dr. Nazar Khan

Matrix and Vector Derivatives

For scalar function $f \in \mathbb{R}$,

$$\nabla_{\mathbf{v}} f = \frac{\partial f}{\partial \mathbf{v}} = \begin{bmatrix} \frac{\partial f}{\partial v_1} & \frac{\partial f}{\partial v_2} & \cdots & \frac{\partial f}{\partial v_D} \end{bmatrix}$$

$$\nabla_{\mathbf{M}} f = \frac{\partial f}{\partial \mathbf{M}} = \begin{bmatrix} \frac{\partial f}{\partial M_{11}} & \frac{\partial f}{\partial M_{12}} & \cdots & \frac{\partial f}{\partial M_{1n}} \\ \frac{\partial f}{\partial M_{21}} & \frac{\partial f}{\partial M_{22}} & \cdots & \frac{\partial f}{\partial M_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial M_{m1}} & \frac{\partial f}{\partial M_{m2}} & \cdots & \frac{\partial f}{\partial M_{mn}} \end{bmatrix}$$

For vector function $\mathbf{f} \in \mathbb{R}^K$,

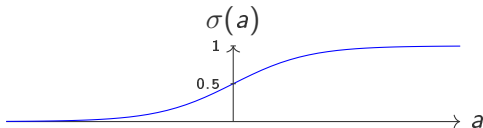
$$\nabla_{\mathbf{v}} \mathbf{f} = \begin{bmatrix} \nabla_{\mathbf{v}} f_1 \\ \nabla_{\mathbf{v}} f_2 \\ \vdots \\ \nabla_{\mathbf{v}} f_K \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial v_1} & \frac{\partial f_1}{\partial v_2} & \cdots & \frac{\partial f_1}{\partial v_D} \\ \frac{\partial f_2}{\partial v_1} & \frac{\partial f_2}{\partial v_2} & \cdots & \frac{\partial f_2}{\partial v_D} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_K}{\partial v_1} & \frac{\partial f_K}{\partial v_2} & \cdots & \frac{\partial f_K}{\partial v_D} \end{bmatrix}$$

Activation Functions

- ▶ Recall that a perceptron has a non-differentiable activation function, i.e., step function.
 - ▶ Zero-derivative everywhere except at 0 where it is non-differentiable.
- ▶ Prevents gradient descent.
- ▶ Can we use a smooth activation function that behaves similar to a step function?
- ▶ Perceptron with a smooth activation function is called a *neuron*.
- ▶ Neural networks are also called multilayer perceptrons (MLP) even though they do not contain any perceptron.

Logistic Sigmoid Function

- ▶ For $a \in \mathbb{R}$, the *logistic sigmoid* function is given by $\sigma(a) = \frac{1}{1+e^{-a}}$
- ▶ *Sigmoid* means S-shaped.
- ▶ Maps $-\infty \leq a \leq \infty$ to the range $0 \leq \sigma \leq 1$. Also called *squashing* function.
- ▶ **Can be treated as a probability value.**
- ▶ Symmetry $\sigma(-a) = 1 - \sigma(a)$. **Prove it.**
- ▶ Easy derivative $\sigma' = \sigma(1 - \sigma)$. **Prove it.**



Activation Functions

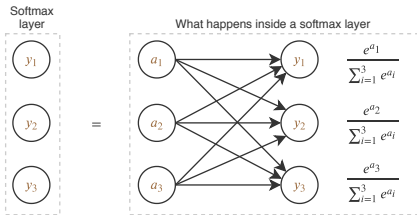
Regression

- ▶ Univariate: use 1 output neuron with identity activation function $y(a) = a$.
- ▶ Multivariate: use K output neurons with identity activation functions $y(a_k) = a_k$.

Classification

- ▶ Binary: use 1 output neuron with logistic sigmoid $y(a) = \sigma(a)$.
- ▶ Multiclass: use K output neurons with *softmax* activation function.

Softmax Activation Function



- For real numbers $a_1, \dots, a_K$, the *softmax* function is given by

$$y(a_k; a_1, a_2, \dots, a_K) = \frac{e^{a_k}}{\sum_{i=1}^K e^{a_i}}$$

- Output of k -th neuron depends on activations of *all neurons in the same layer*.

Softmax Activation Function

- ▶ Softmax is ≈ 1 when $a_k \gg a_j \forall j \neq k$ and ≈ 0 otherwise.
- ▶ Provides a smooth (differentiable) approximation to finding the *index of the maximum element*.
 - ▶ Compute softmax for 1, 10, 100.
 - ▶ Does not work everytime.
 - ▶ Compute softmax for 1, 2, 3. Solution: multiply by 100.
 - ▶ Compute softmax for 1, 10, 1000. Solution: subtract maximum before computing softmax.
- ▶ Also called the *normalized exponential* function.
- ▶ Since $0 \leq y_k \leq 1$ and $\sum_{k=1}^K y_k = 1$, *softmax outputs can be treated as probability values*.
- ▶ Show that $\frac{\partial y_k}{\partial a_j} = y_k(\delta_{jk} - y_j)$ where $\delta_{jk} = 1$ if $j = k$ and 0 otherwise.

Summary

- ▶ Inputs and outputs can be univariate as well multivariate.
- ▶ Regression requires SSE loss.
- ▶ Classification requires cross-entropy loss.
- ▶ Minimization corresponds to finding the point where derivate of loss is zero.
- ▶ Matrix and vector calculus makes calculations and computation cleaner and faster.
- ▶ Next lecture: Training MLPs.